



Bronco Robotics Exploration (BREx)

Autonomous Navigation and Crash Avoidance

Team: Eduardo Guerrero, Mariana Rivera, Wesley Hwee, Hao Guo, Jisko

Alvarez IV, Anthony Ojeda, Alan Castillo

Date: 05/12/2026



College of Engineering
ELECTROMECHANICAL ENGINEERING TECHNOLOGY

TABLE OF CONTENTS

Abstract.....	4
1. Introduction.....	5
1.1 Objective.....	5
2. Passive Wiring Board.....	7
2.1 Design Motivation.....	7
2.2 Board Architecture, Integration, and Improvements.....	7
3. Mezo Level Controller.....	16
3.1 Introduction.....	16
3.2 Hardware Architecture Brief.....	16
3.3 STM32 Nucleo F767ZI Feedback Controller.....	17
3.4 Motor Controller.....	19
3.5 Safety Timeout Behavior.....	20
3.6 Feedback Packet Reporting.....	22
3.7 Heading Computation.....	23
3.8 Data Collection and Testing.....	24
4. Autonomous Navigation.....	26
4.1 Introduction.....	26
4.2 Localization and Sensor Fusion.....	27
4.3 GNSS Integration and Coordinate Transformation.....	28
4.4 Manual Route Recording and Autonomous Replay.....	29
4.5 Rolling Costmap and Incremental Navigation.....	30

4.6 Testing and Validation..... 31

4.7 Reflection.....33

5. Crash Avoidance Behavior..... 34

5.1 Semantic Segmentation and Environmental Perception.....34

5.2 Visualization_node..... 36

5.3 Operator camera view.....36

5.4 Testing and integration support..... 36

6. Conclusion.....38

Appendix B. Works Cited..... 39

Abstract

This BReX spiral focused on improving the existing iHerb robotic vehicle by developing a more reliable autonomous navigation and crash avoidance system. The project expanded on previous team designs by redesigning the passive wiring board, improving motor driver integration, implementing meso-level control architecture, and developing ROS2-based autonomous driving capabilities. A new PCB design was created to reduce wiring complexity, eliminate unreliable jumper connections, and improve the physical stability of the motor driver and microcontroller interface. In addition, a Colibri iMX7-based meso-level controller was introduced to separate low-level motor control from high-level processing on the NVIDIA Jetson Orin.

The robot's autonomous behavior was developed through the integration of sensor fusion, localization, route recording, and command-based motor control. GPS, IMU, and wheel encoder data were combined using ROS2 localization tools to produce a more reliable estimate of the robots position, velocity, and orientation. The system was also updated to support standard ROS2 `/cmd_vel` commands, allowing the robot to operate in both manual joystick mode and autonomous mode. Safety visualizations were added to improve reliability during testing. Overall, this project improved the robot's hardware organization, control architecture, and autonomous navigation capabilities while providing a foundation for future development.

1. Introduction

1.1 Objective

The objective of this year's BREx project was to improve upon the previous teams' development of the iHerb robotic vehicle by strengthening the robot's motor control, hardware integration, autonomous navigation, and sensor fusion capabilities. Previous work established the foundation for the robot's movement and control system, but additional improvements were needed to make the platform more reliable and capable of autonomous operation.

A major goal was to improve the connection between the motor driver and microcontroller system by redesigning the PCB and reducing the need for loose jumper cables. This helped create a more stable and organized hardware interface for testing and operation. Another objective was to improve the robot's control architecture by separating low-level motor execution from higher-level autonomous behavior. This allowed the robot to support both manual joystick control and autonomous velocity commands through ROS2.

The project also focused on developing the robot's sensor fusion and localization system. Using ROS2, the robot was configured to read and process sensor information from GPS, IMU, and motor encoder feedback. Since each sensor has limitations when used alone, the sensor data was combined to produce a more reliable estimate of the robot's position, orientation, and movement. This fused localization data became an important foundation for autonomous navigation, route replay, and future waypoint-following behavior.

Overall, the objective of the project was to move the iHerb robot closer to full autonomous navigation by improving the reliability of its hardware, expanding its control

system, and integrating sensor data through ROS2-based localization and sensor fusion. This year's work provides a stronger platform for future teams to continue developing autonomous driving, crash avoidance and advanced navigation behaviors.

2. Passive Wiring Board

2.1 Design Motivation

The PCB was designed with two main goals: replicating the functionality of the original iHerb robot board and reducing the amount of wiring and jumper cables required between the PCB and the microcontroller.

2.2 Board Architecture, Integration, and Improvements

Board architecture was inspired by the original PCB design, the interaction of the motor driver with the microcontroller. Using the architecture of the original board, one of the original teams working on the iHerb attempted to design a custom PCB with the functionality of the original.

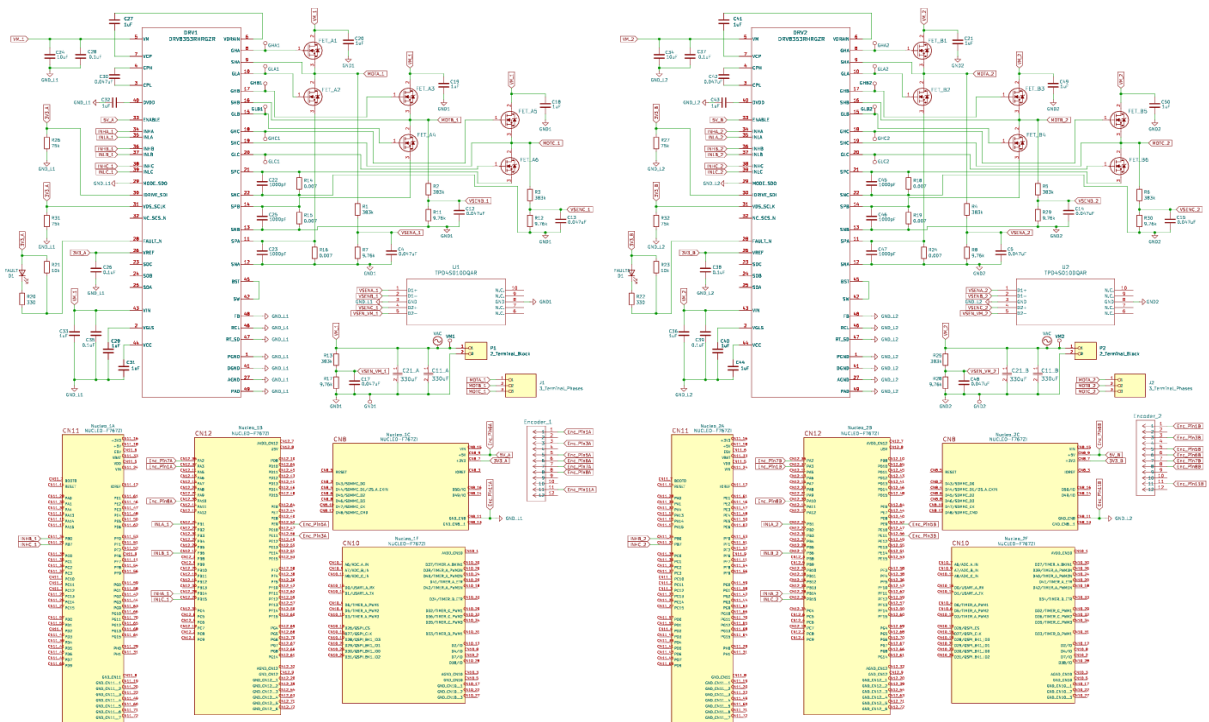


Figure 1. Schematic made by the original team

Although testing with this board yielded favorable results, there were problems encountered with the design. For one, due to the design of the board, it uses majority through-hole components that caused the board to be larger. Although not a major issue, it was larger than the original PCB so mounting the board proved a problem. Another major design flaw of the board was that it wasn't tested thoroughly beforehand and had trace/connection problems that would not have been detected until later testing and further investigation.

This issue was resolved by the subsequent team. Rather than designing a motor driver board from scratch using the original iHerb board as a reference, they opted to purchase a premade motor driver board from Texas Instruments (TI), eliminating the design challenges the previous team encountered. They also redesigned the PCB to serve as a mounting platform for the STM32 Nucleo-F767ZI microcontroller. The boards were arranged so that the motor driver used female header pins and the microcontroller PCB used male header pins, allowing the two boards to be connected with jumper cables.



Figure 2. View of header pins

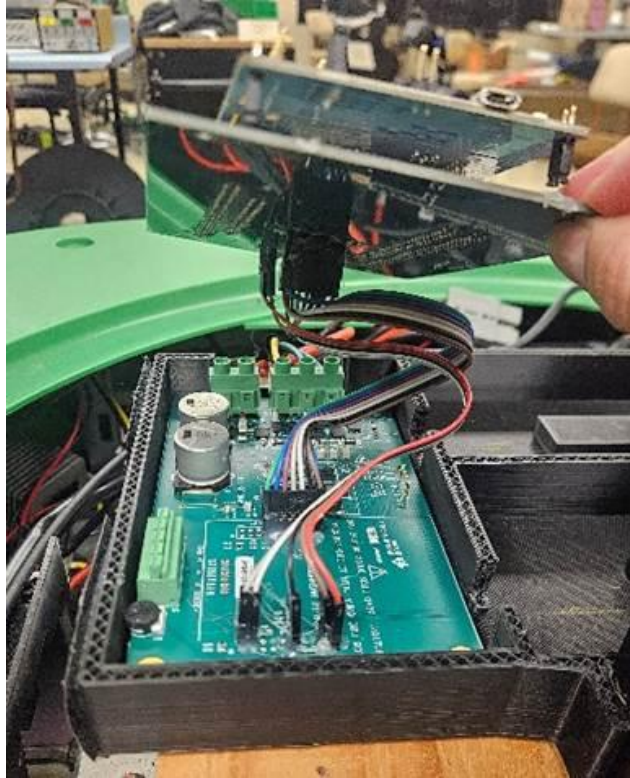


Figure 3. View of bottom cabling

The testing and demonstration of the iHerb using this design were successful. However, two issues with the boards became apparent. First, because the system relied on jumper cables, the motor driver board and the microcontroller PCB had to be positioned directly next to each other. This reintroduced the original problem of the PCB occupying too much space on the robot. Second, the jumper cables were prone to disconnecting during testing and demonstrations, especially when the robot operated on rough terrain.

At this point, the PCB team for this year chose to perform a complete redesign and re-documentation of the PCB, referencing both the original board and the version created by the previous team. As part of the documentation effort, we traced every wire, cable, and PCB trace from both designs and compiled the information into two separate Excel files. From there, we

cross-referenced all connections to determine where each signal originated and terminated, ultimately producing a new, consolidated Excel document.

Nucleos Pin Connections:									
CN#	PIN	Type	Connected To	PIN	Type	Description			
CN11	PB0		MMDPMFET	36	INHB_1			Device/IC Name:	Acronym
CN11	PB7		MMDPMFET	38	INHC_1			Multiphase Motor Driver Power MO	MMDPMFET
CN12	PA2		Encoder_1	Enc_Pin7A	Hall_V			Common Ground	CGND
CN12	PA3		Encoder_1	Enc_Pin1A	Hall_U				
CN12	PA10		Encoder_1	Enc_Pin8A	Hall_W				
CN12	PB1		MMDPMFET	35	INLA_1				
CN12	PB6		MMDPMFET	37	INLB_1				
CN12	PB14		MMDPMFET	34	INHA_1				
CN12	PB15		MMDPMFET	39	INLC_1				
CN12	PE9		Encoder_1	Enc_Pin5A	Encoders	*May have to switch with PE11			
CN12	PE11		Encoder_1	Enc_Pin3A	Encoders	*May have to switch with PE9			
CN8	CN8_3	5V	Encoder_1PWR	Enc_Pin6 5V_A	PWR				
CN8	CN8_7	3V3	MMDPMFET	30	Non-specific/Several				
CN8	CN8_11	GND	Encoder_1CGND	Enc_Pin11A GND	GND_L1				
Multiphase Motor Driver Power MOSFET (MMDPMFET/DRV8353RHRGZR) Pin Connections									
PIN	Type	Connected To	PIN	Type	Description				
5	VM	PWR	VM_1						
7	VCP	MMDPMFET	6		VDRAIN				
4	CPH	MMDPMFET	3		CPL				
3	CPL	MMDPMFET	4		CPH				
40	DVDD	CGND	GND						
33	Enable	Nucleos	CN8_7	3V3					
34	INHA	Nucleos	PB14	INHA_1					
35	INLA	Nucleos	PB15	INLA_1					
36	INHB	Nucleos	PB16	INHB_1					
37	INLB	Nucleos	PB17	INLB_1					
38	INHC	Nucleos	PB18	INHC_1					
39	INLC	Nucleos	PB19	INLC_1					
29	GND	GND_L1	PB20						
30	IDRIVE_SC	GND_L1	PB21						
31	VDS_SCL	GND_L1	PB22						
28	Fault_N	Nucleos	PB23	3V3					
26	VREF	Nucleos/GND_L1	PB24	3V3/GND					
43	VIN	PWR/GND_L1	PB25	VM_1					
2	VGLS	GND_L1							
44	VCC	GND_L1							
6	VDRAIN	MMDPMFET	5	VM					
8	GHA	FET_A1	1	GATE					
9	SHA	Terminal Phases	1	MOTA_1					
10	GLA	FET_A2	1	GATE					
17	GHB	FET_A3	1	GATE					
16	SHB	Terminal Phases	2	MOTB_1					
15	GLB	FET_A4	1	GATE					
18	GHC	FET_A5	1	GATE					
19	SHC	Terminal Phases	3	MOTC_1					
20	GLC	FET_A6	1	GATE					
21	SPC	FET_A6	3	DRAIN					
22	SNC	TPD4S010DQAR	4	D2+					
14	SPB	FET_A4/TPD4S010DG	3 2	GATE/D1-					
13	SNB	TPD4S010DQAR	2	VSENB_1					
11	SPA	FET_A2/TPD4S010DG	3 1	GATE/VSENA_1					
12	SNA	TPD4S010DQAR	1	VSENA_1					
45	BST	MMDPMFET	42	SW					
42	SW	MMDPMFET	45	BST					
48	FB	GND_L1							
46	RCL	GND_L1							
47	RT_SD	GND_L1							
1	PGND	GND_L1							
41	DGND	GND_L1							
27	AGND	GND_L1							
49	PAD	GND_L1							

Figure 4. Pinout for the PCB

From this work, the first iteration of the new PCB was developed. Building on the previous team's approach, using the TI motor driver board and creating a dedicated PCB for the

[illegible]

Figure 5. Schematic of PCB

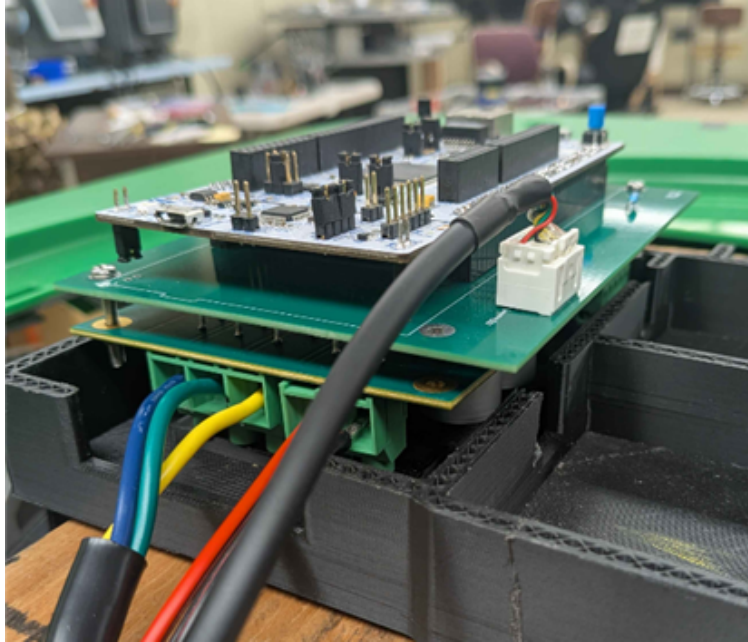


Figure 6. PCB and motor driver mounted

This design became the version used for the testing and demonstration of the iHerb robot. A further redesign was considered, one that would eliminate the need for header pins entirely by connecting the motor driver and microcontroller PCB directly through board-to-board traces. However, due to development time constraints, PCB manufacturing and delivery timelines, and the project's overall objectives, this redesign was not pursued.

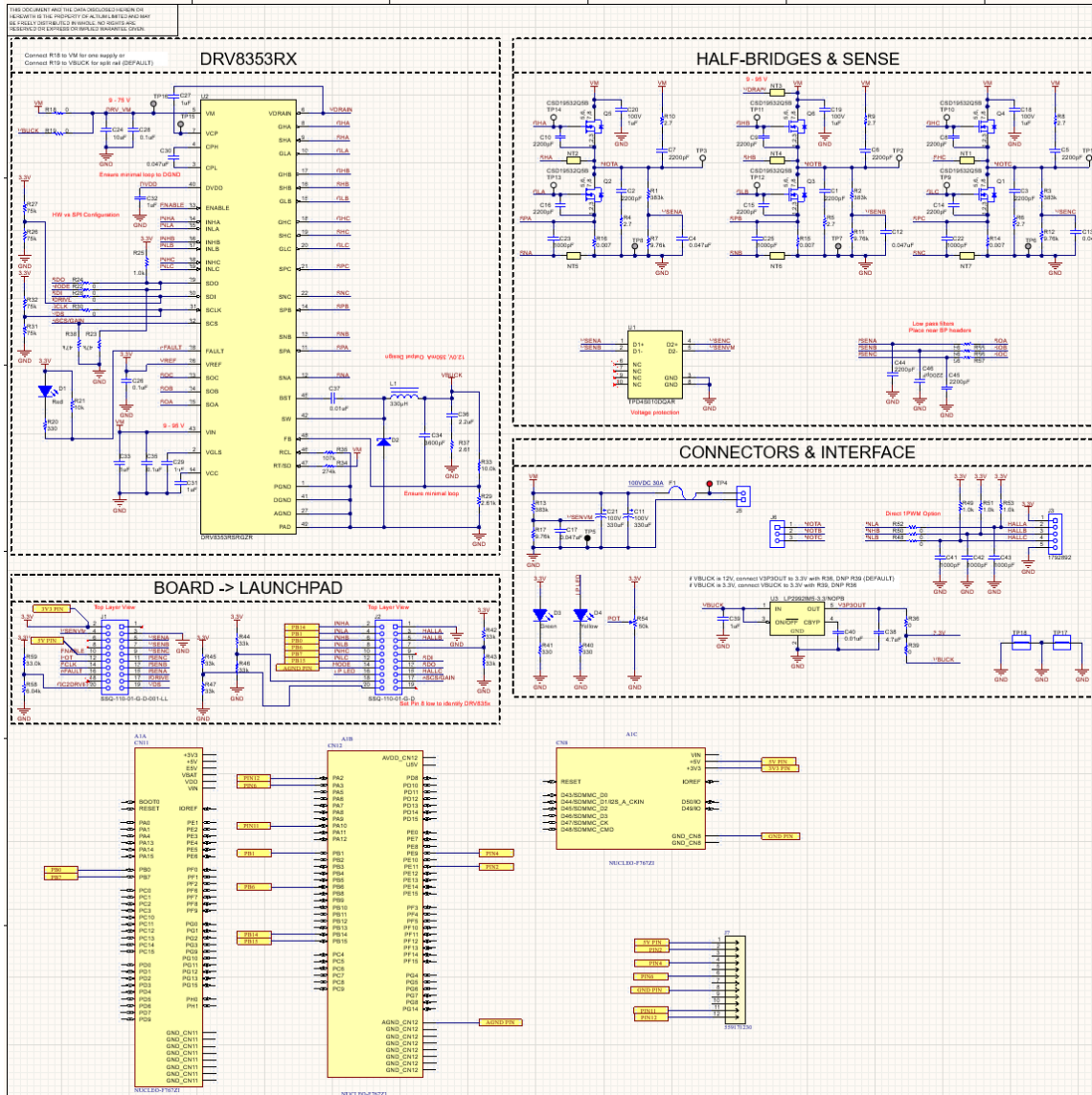


Figure 7. Schematic of the second iteration redesign

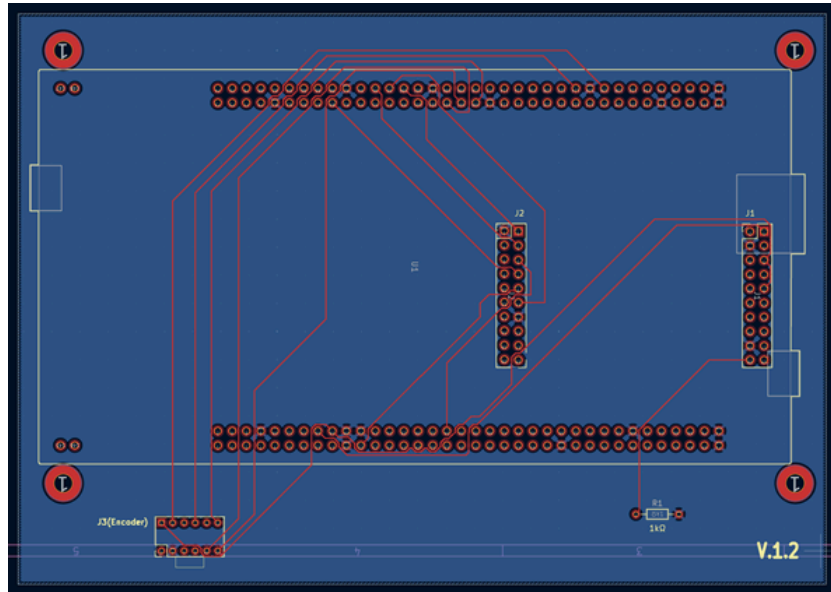


Figure 8. Layout of current PCB (V1.2)

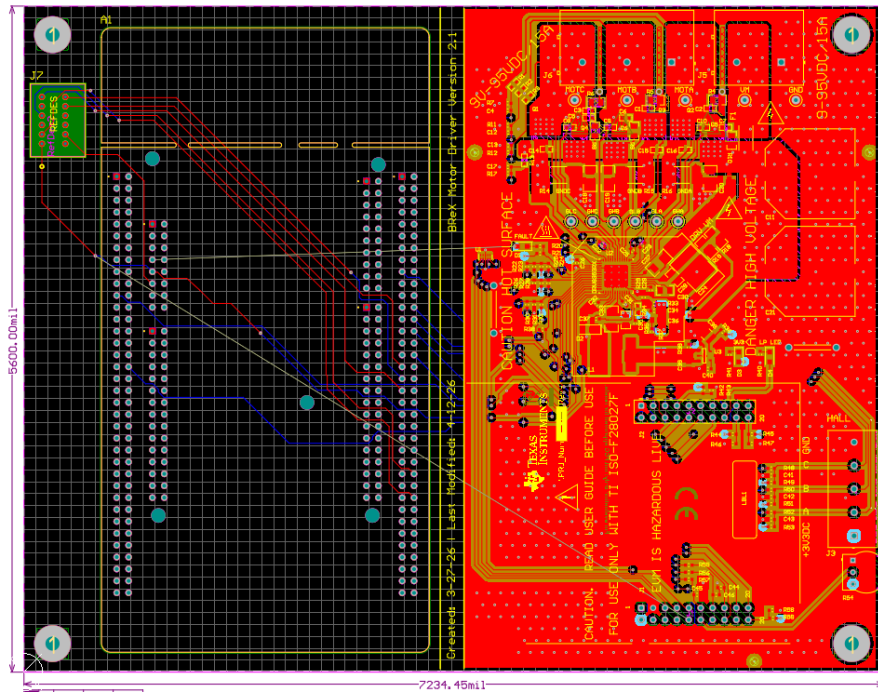


Figure 9. Layout of redesigned PCB (V2.2)

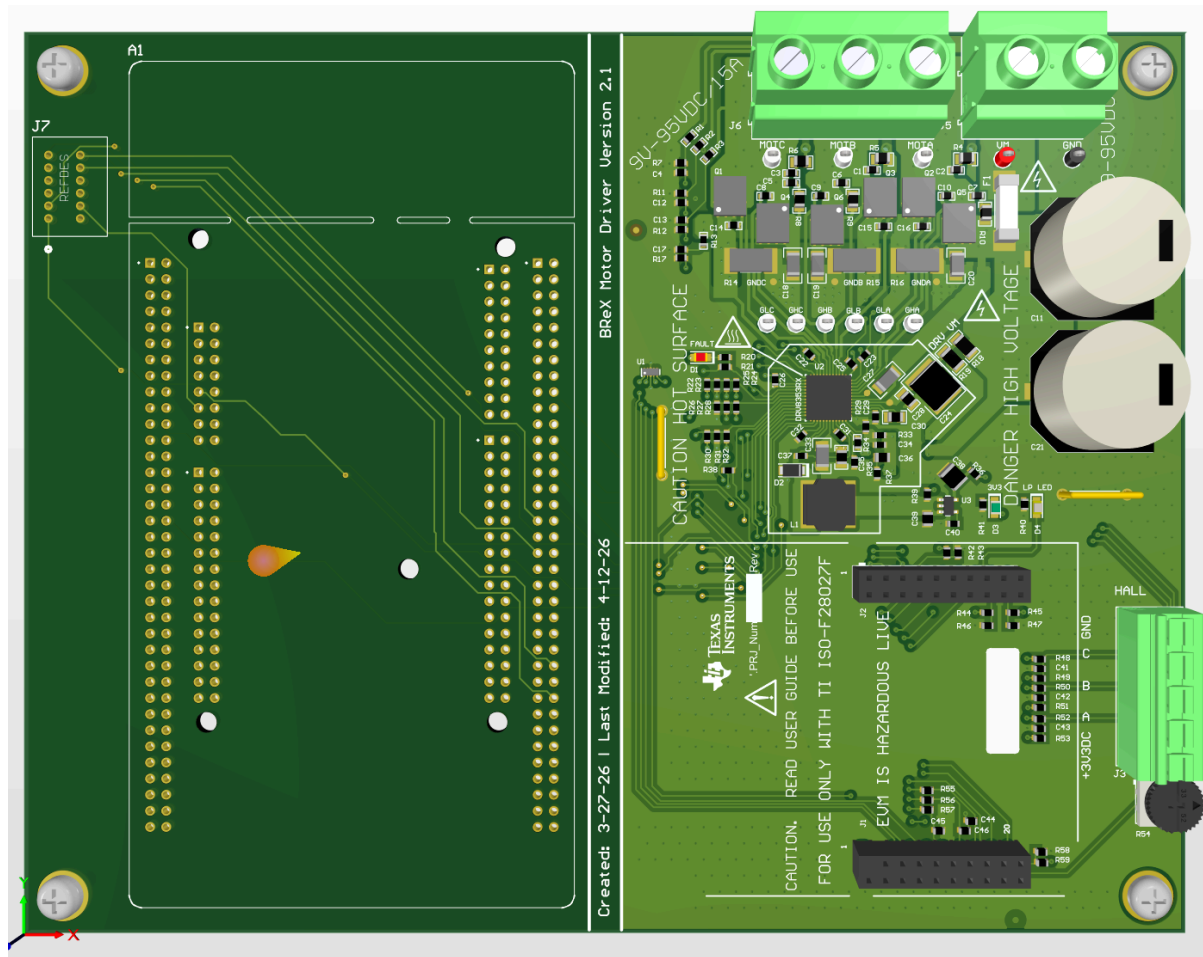


Figure 10. 3D Layout of redesigned PCB (V2.2)

3. Mezo Level Controller

3.1 Introduction

The Mezo level architecture design and purpose is to separate low-level motor execution and higher-level behaviors including navigation, scene segmentation and sensor fusion. The mezo level controller will offload the original joystick only serial motor controller from the Orin and be implemented as a new more complete motor controller that supports both manual joystick commands and autonomous `/cmd_vel` driving onto the new computer module, the Colibri iMX7. Thus, leaving the Orin to focus on higher level behavior processing and offering stability and reliability of the motor controller making driving more stable and deterministic while lessening its vulnerability to the Orin.

3.2 Hardware Architecture Brief

The new Mezo level hardware architecture will consist of the Iris Carrier development board from Toradex and the iMX7 computer module. Velocity commands will be sent from the Orin to the iMX7 via ethernet cable over `ROS_DOMAIN_ID` local network and then output those commands to the left and right motor driver STM32 nucleos via UART. The feedback and heading will thus be reported back over UART to the Colibri iMX7 and displayed on the Orin's terminal. The Colibri iMX7 is also equipped with a micro SD card in order to expand its onboard storage and allow for a larger ROS 2 and Docker workspace.

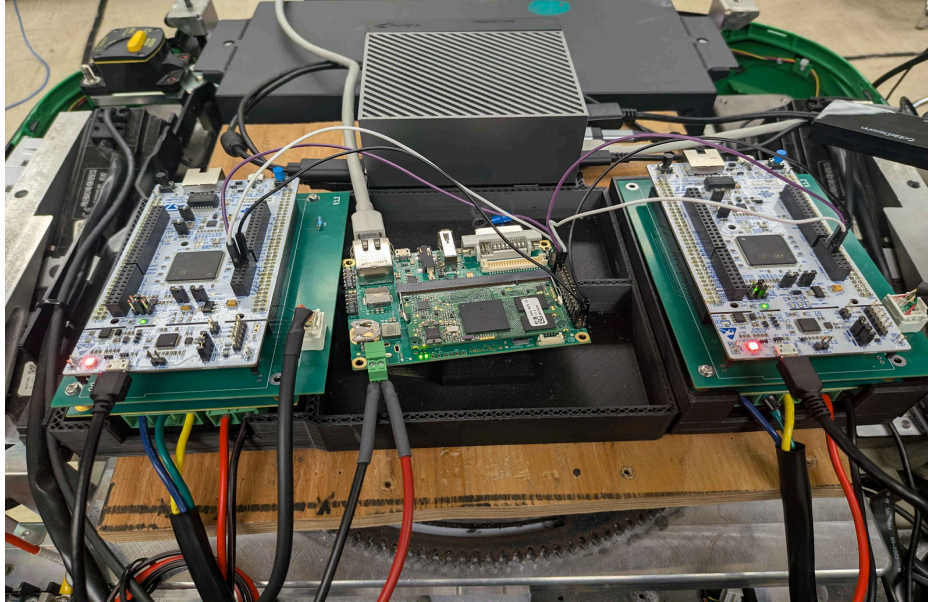


Figure 11. iMX7 Iris carrier board connected to the Orin and Nucleos

3.3 STM32 Nucleo F767ZI Feedback Controller

Adding feedback , updates to original code were made as the Nucleos communicated with the Orin directly via MicroUSB (USART3/huart3) . With the inclusion of the iMX7, the Nucleo IOC file had to be reconfigured to enable UART5 (huart5). The Callback function was mostly kept the same as the only command lines that needed to be changed were the lines that would receive data from huart3 to receiving data from huart5 ports instead. The Nucleos will be using ports PD2 and PC12 to communicate with the iMX7.

```

void HAL_UART_RxCpltCallback(UART_HandleTypeDef*huart)
{
    // if (huart->Instance == USART3) commented out by Mariana to change to Imx7
    if (huart->Instance == UART5)
    {
        static char rx_buffer[50];
        static uint8_t rx_index = 0;

        if (rx_data[0] != '\n' && rx_index < sizeof(rx_buffer) - 1)
        {
            rx_buffer[rx_index++] = rx_data[0];
        }
        else
        {
            rx_buffer[rx_index] = '\0';
            rx_index = 0;

            char sign;

            if (sscanf(rx_buffer, "V=%k%lu", &sign, &velocity_ref) == 2)
            {
                if (velocity_ref < 0) velocity_ref = 0;
                if (velocity_ref > 300) velocity_ref = 300;

                // Update direction based on sign
                if (sign == '+')
                    Direction = 1; // Forward
                else if (sign == '-')
                    Direction = 0; // Backward
                else
                    Direction = 1; // default to forward if unexpected
            }

            // char msg[50];
            snprintf(msg, sizeof(msg), "V updated to %lu \r\n", velocity_ref);
            // HAL_UART_Transmit(&huart3, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY); //commented out by mariana to change to imx7 uart5
            HAL_UART_Transmit(&huart5, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
            last_command_time = HAL_GetTick();
        }
    }
}

```

Figure 12. Adjustments to the Nucleo's Callback Function

The one addition to this function was reporting velocity feedback to the iMX7 to allow the computation of the heading control that was moved to the iMX7. The feedback function will report back the wheel velocity, velocity reference, the current period, and its direction.

```

    }
    else if (strcmp(rx_buffer, "FB") == 0)
    {
        Send_Feedback_UART5();
        last_command_time = HAL_GetTick();
    }
    else
    {
        char msg[] = "Invalid format. Use V=xx\r\n";
        // HAL_UART_Transmit(&huart3, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY); //commented out to change to imx7 uart5 by mariana
        HAL_UART_Transmit(&huart5, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
    }
}

// Re-arm UART interrupt
// HAL_UART_Receive_IT(&huart3, (uint8_t *)rx_data, sizeof(rx_data)); //commented out to change to imx7 uart5 by mariana
HAL_UART_Receive_IT(&huart5, (uint8_t *)rx_data, sizeof(rx_data));
}
}

```

Figure 13. Addition of the Velocity Feedback in the Call Back function

```

/* USER CODE BEGIN 4 */

void Send_Feedback_UART5(void)
{
    char fb_msg[80];

    snprintf(
        fb_msg,
        sizeof(fb_msg),
        "FB,%ld,%ld,%d,%lu\r\n",
        (long)velocity_current,
        (long)velocity_ref,
        Direction,
        (unsigned long)current_period
    );

    HAL_UART_Transmit(&huart5, (uint8_t *)fb_msg, strlen(fb_msg), HAL_MAX_DELAY);
}

```

Figure 14. Feedback Function that's been added

3.4 Motor Controller

In the previous iteration, the iHerb robotic vehicle was incapable of driving autonomously and relied only on joystick controller commands. Our new implementation of the controller introduced the new ROS 2 message type `Twist` and the node subscription `'cmd_vel'` where the iHerb robot can now accept standard ROS velocity commands for autonomous driving. We still wanted to keep the joystick commands for human interception so our new controller introduces a switch case scenario and the ability to track whether it is in joystick, autonomous, or idle mode.

The ROS 2 message type `Twist` is a standard velocity command message and expresses velocity in a free space broken into its linear and angular parts. With the new updated controller, the node subscribes to `/cmd_vel` and receives a `Twist` message containing the desired forward speed and turning rate. We then implemented two new functions to convert the messages into left and right wheel speeds.

Our first function, `twist_callback()`, runs whenever a `Twist` message has arrived. The most recent command is stored in `self.last_autonomous_twist` and records the time it arrived

in `self.last_autonomous_time` to keep track of the most recent. The second function, `_twist_to_wheel_speeds()` converts the linear and angular parts into motor commands. It reads the forward and backward speed from `twist.linear.x` and the turning speed from `twist.angular.z` and calculates each wheel's target speed using differential drive kinematics. After calculating the wheel speeds, the function converts them into the controller's scaled integer values so they do not exceed the capped `max_speed`.

```
def twist_callback(self, msg):
    self.last_autonomous_twist = msg
    self.last_autonomous_time = self.get_clock().now()

def _twist_to_wheel_speeds(self, twist):
    linear = twist.linear.x
    angular = twist.angular.z
    left_mps = linear - angular * (self.wheel_base_m * 0.5)
    right_mps = linear + angular * (self.wheel_base_m * 0.5)
    vl_speed = int(round(left_mps / self.feedback_velocity_scale))
    vr_speed = int(round(right_mps / self.feedback_velocity_scale))
    vl_speed = max(-self.max_speed, min(vl_speed, self.max_speed))
    vr_speed = max(-self.max_speed, min(vr_speed, self.max_speed))
    return vl_speed, vr_speed
```

Figure 15. Twist conversion functions for wheel speed

3.5 Safety Timeout Behavior

Our newest controller iteration introduced new safety timeout parameters and behaviors to make sure the iHerb vehicle does not continuously move when command messages stop arriving. By adding the new parameters for joy timeout and `cmd_vel` timeout, the code looks for commands every 0.5 seconds after the last message was received.

A timer was implemented to complement the timeout parameter that runs every 0.1 seconds to check whether the vehicle is receiving valid control input commands. When

joystick commands stop for longer than `joy_timeout_s`, the code then checks whether there was a recent `/cmd_vel` command by comparing the current time to `self.last_autonomous_time`. If the latest was an autonomous command and has not surpassed `cmd_vel_timeout_s`, the robot switches to autonomous control. If neither joystick input nor `/cmd_vel` is recent, the new iteration of code switches modes to idle and sends out a command of (0,0) stopping both motors. This was an important update to make sure the iHerb vehicle does not continue moving and improves safety overall.

Our code also made sure to account for possible joy controller defects like stick drift by implementing a deadzone. If the axes are close to zero for the joystick, the code sends zero speed commands (0,0) and logs the joystick in deadzone. This prevented any small drift from accidentally moving the vehicle.

```
def check_joy_timeout(self):
    now = self.get_clock().now()
    joy_age = (now - self.last_joy_time).nanoseconds * 1.0e-9

    if joy_age <= self.joy_timeout_s:
        return

    if self.last_autonomous_twist is not None:
        cmd_age = (now - self.last_autonomous_time).nanoseconds * 1.0e-9
        if cmd_age <= self.cmd_vel_timeout_s:
            vl_speed, vr_speed = self.twist_to_wheel_speeds(self.last_autonomous_twist)
            if self.current_drive_mode != 'autonomous':
                self.get_logger().info('Switched to autonomous drive mode')
                self.current_drive_mode = 'autonomous'
            if [vl_speed, vr_speed] != self.last_drive_cmd:
                self.get_logger().info(
                    f'Autonomous active: VL={vl_speed} VR={vr_speed}'
                )
            self._send_wheel_commands(vl_speed, vr_speed)
            return

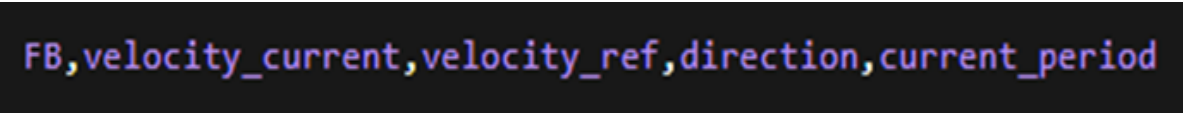
    if self.last_drive_cmd != [0, 0]:
        if self.current_drive_mode != 'idle':
            self.get_logger().info('Switched to idle mode')
            self.current_drive_mode = 'idle'
        self.get_logger().warn(
            f'No joystick or cmd_vel data, stopping motors'
        )
        try:
            self._send_wheel_commands(0, 0)
```

Figure 16. Snippet of Timeout Code

3.6 Feedback Packet Reporting

From the iMX7 to the Nucleo, it would request the feedback data received in the format of “FB,velocity_current,velocity_ref,direction,current_period” (look at Figure 18). Velocity reference (300 cap) and the current velocity can be compared to indicate whether the motor is going at the speed of the desired velocity given by the Orin. We can also see the direction of the wheel for verification if the wheel is supposed to be spinning in the correct direction. Finally, the current period is reported to track the pulse interval that the motor is receiving for debugging reasons. So if one tells the motor to move slowly, you would expect a large value and vice versa. Most of the data of the feedback is mostly used for debugging in case the robot is not behaving how it's supposed to be.

The current velocity is taken for computation of the yaw for the odometry that can be seen graphically and visually (for debugging). The yaw is also converted to quaternions for the heading that is reporting back to the Orin. More details on these calculations can be read in the next section. Figure 21 is an example of what the iMX7 is reporting back to the Orin for heading feedback.



```
FB,velocity_current,velocity_ref,direction,current_period
```

Figure 17. Feedback Format

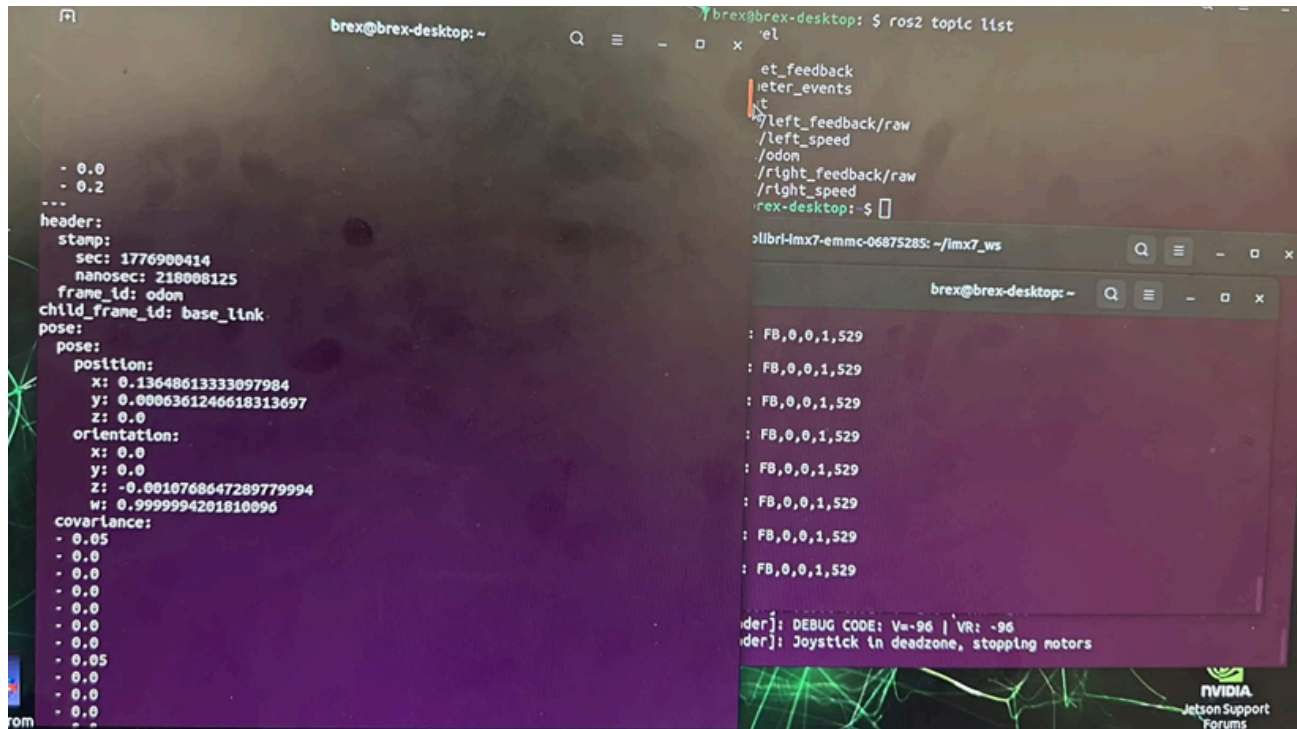


Figure 18. Visual Reportings of Feedback to the Orin

3.7 Heading Computation

With the new feedback packet in place, the iMX7's motor controller actively requests feedback from the left and right wheel controllers and reports the heading computation to the Orin. The controller code extracts the velocity feedback from each STM32. It then applies the direction value from the packet to determine whether it is moving forward (positive) or backward (negative) and uses the `set_velocity()` function to scale the raw feedback into meters per second.

Position of the vehicle is calculated using velocities over time to update the x, y and yaw. `Publish_wheel_odom()` calculates the robot's angular (yaw rate) and it updates the heading by time integration using the generic equation $\text{yaw_new} = \text{yaw_old} + \text{angular} * \text{dt}$ (`self.yaw += angular * dt`), where dt is the elapsed time since the last odometry update. This

allows the heading to continuously update by accumulating the small changes in rotation over time. The new yaw is then converted into a quaternion where ROS can then publish orientation inside the /wheel/odom message by the function yaw_to_quaternion(yaw).

```
linear = (self.left_velocity + self.right_velocity) * 0.5
angular = (self.right_velocity - self.left_velocity) / self.wheel_base_m

self.x += linear * math.cos(self.yaw) * dt
self.y += linear * math.sin(self.yaw) * dt
self.yaw += angular * dt
```

Figure 19. Yaw calculation code for odometry

```
def yaw_to_quaternion(yaw):
    quat = Quaternion()
    half_yaw = yaw * 0.5
    quat.z = math.sin(half_yaw)
    quat.w = math.cos(half_yaw)
    return quat
```

Figure 20. Yaw to Quaternion calculation

3.8 Data Collection and Testing

With the new updated motor controller and feedback parser, the iMX7 controller outputs the data while the vehicle is in motion to monitor the robot's live ROS 2 feedback topics. Using rqt_plot, the Orin was able to display real time graphs of the wheel speed for right and left and the odometry data of angular vs. linear reported from the iMX7 from the STM32 nucleos. This allowed our team to confirm that the feedback packets were being received, parsed, scaled and published. The live plots allowed us to verify that the left and

right wheel speeds responded to drive command inputs and turning commands produced an expected difference between wheel velocities. The odometry graph also allowed us to verify odometry was updating continuously during vehicle motion.

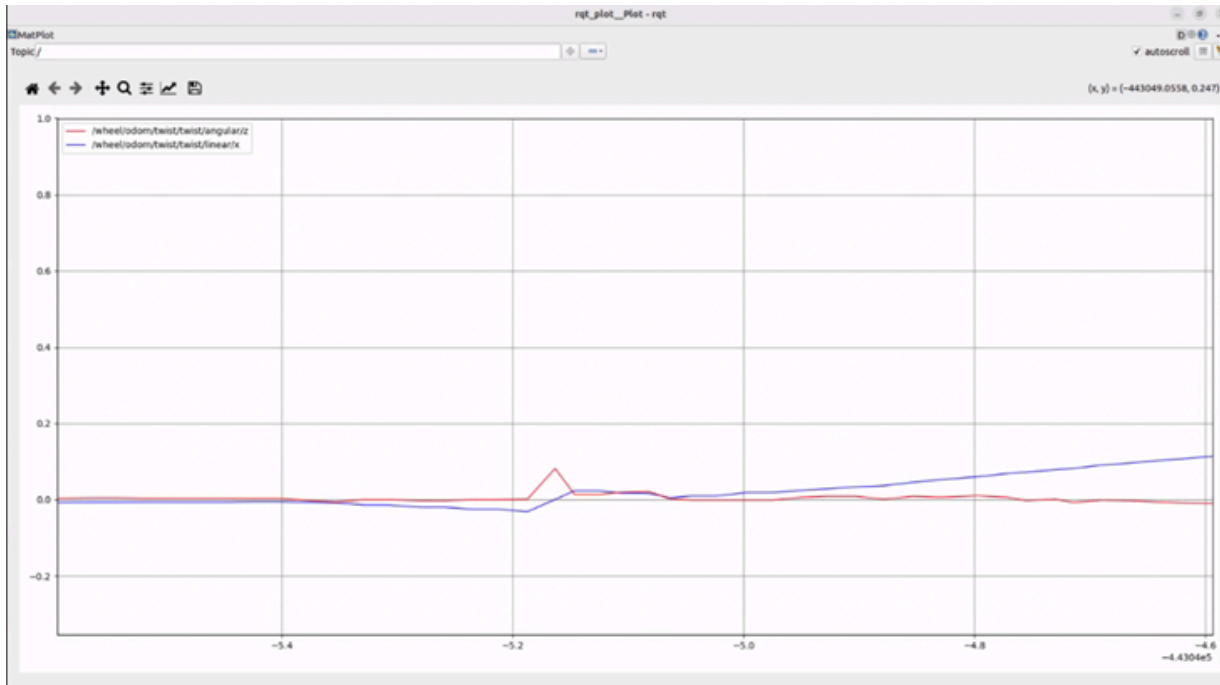


Figure 21. Vehicle Angular and Linear Graph

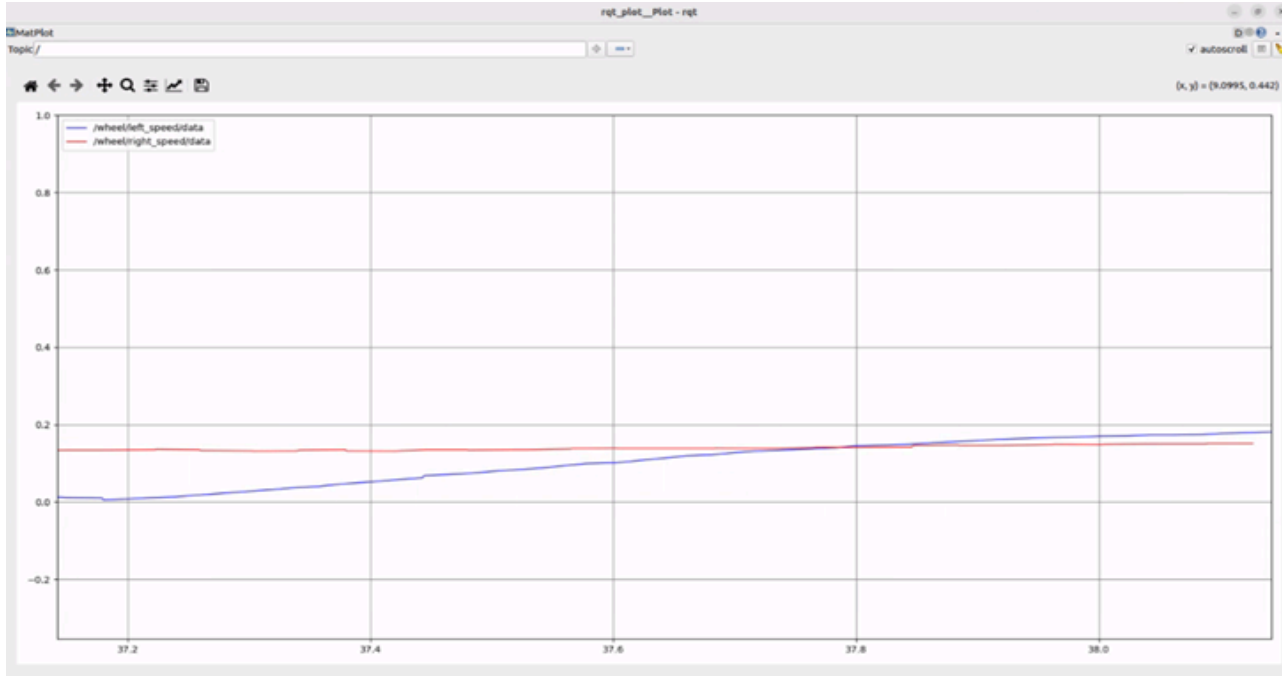


Figure 22. Left and Right Wheel Speed Data

4. Autonomous Navigation

4.1 Introduction

In this section, we will go over our main goal of our spiral, which was to achieve autonomous navigation. Our system was developed to allow the iHerb robot to move beyond manual control and begin following recorded routes autonomously. The primary focus of this work was localization, sensor fusion and autonomous route replay using GPS breadcrumbing. The system was designed to provide the robot with a reliable estimate of its position and orientation while integration with the previous team's architecture.

The navigation framework was made with ROS2 on the Jetson Orin, and we combined multiple sensor inputs into a single filtered localization estimate that could be used for autonomous navigation and waypoint following.

4.2 Localization and Sensor Fusion

A challenge one faces in autonomous navigation and using sensors is that no single sensor is completely reliable. They can't provide a reliable estimate of the robot's position. GPS data gives a globally referenced positioning but can become noisy or unreliable. IMU data gives orientation and acceleration measurements but it is susceptible to drifting over time. Encoder feedback from the motors can estimate how far the robot has traveled, but it can also accumulate error due to wheel slip, uneven terrain, or mechanical inaccuracies.

To improve localization accuracy, the robot uses sensor fusion. Sensor fusion combines data from multiple sensors to create a more reliable estimate of the robot's position and movement. In this project, GPS, IMU, and wheel encoder data are fused together using the `robot_localization` package in ROS2. This package incorporated an Extended Kalman Filter to estimate the robot's state, including position, velocity, and orientation. The filter works by predicting the robot's motion based on previous data and then correcting that prediction using new sensor measurements.

This approach allows the system to reduce the weaknesses of individual sensors. GPS provides global positioning, the IMU helps track orientation and motion changes, and the encoders provide local movement feedback from the motors. By combining these sources, the robot can maintain a more stable and accurate estimate of its location. This fused localization data is then used by the navigation system to help the robot make movement decisions and follow a planned path more reliably.

4.3 GNSS Integration and Coordinate Transformation

```
ekf_filter_node:
  ros__parameters:
    frequency: 30.0
    sensor_timeout: 0.1
    two_d_mode: true

    map_frame: map
    odom_frame: odom
    base_link_frame: base_link
    world_frame: odom

    publish_tf: true

    odom0: /odom
    imu0: /imu/data
```

Figure 23: from ekf.yaml; EKF config. For Sensor Fusion

After the Extended Kalman Filter was configured, the GNSS data was integrated into the localization system with the *navsat_transform_node* from ROS2. This combines the GPS data with IMU orientation and filtered odometry information so we can convert latitude and longitude coordinates into a local map frame that can be used by Nav2. Since Nav2 doesn't directly navigate using raw GPS coordinates, this transformation step is important for autonomous outdoor navigation.

```
Node(
  package='robot_localization',
  executable='navsat_transform_node',
  name='navsat_transform_node',
  parameters=[params],
  remappings=[
    ('gps/fix', gps_topic),
    ('imu', imu_topic),
    ('odometry/filtered', odom_topic),
  ],
)
```

Figure 24: From navsat_manual.yaml; Manual Datum Config. For Indoor Testing

The localization system was configured to support both indoor and outdoor testing. During the outdoor testing, the live GNSS data was used to give real-world positioning info. For indoor

testing, a manually defined GPS point was implemented to simulate a GPS reference point when satellite signals were unavailable. This allows the localization pipeline and coordinate transformations to still be tested without relying on live GPS data.

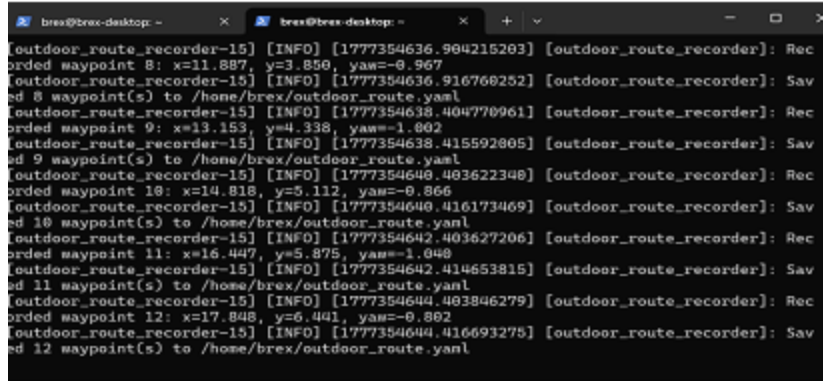
```
navsat_transform_node:  
  ros__parameters:  
    wait_for_datum: true  
    datum: [34.0596697, -117.822119, 0.0]
```

Figure 25: Manual Datum Config. For Indoor Testing; located in navsat_manual.yaml

4.4 Manual Route Recording and Autonomous Replay

The robot operated in two main modes: manual mode and autonomous mode. In manual mode, the robot was driven using a controller while the localization system continuously collected sensor and pose data in the background. This allowed us to verify that important ROS2 topics such as GNSS, Odometry, filtered Odometry, and the coordinate transform tree (*/tf*) were published correctly during operation.

During manual mode, waypoint positions were recorded into a .yaml file for larger autonomous replay. Each waypoint stored position and heading information that could be used by the navigation system.

A terminal window with a dark background and white text. It shows a series of log messages from a process named 'outdoor_route_recorder-15'. The messages are grouped by brackets and include an 'INFO' level log with a timestamp [1777354636.984215203]. Each group contains a 'Recorded waypoint' message with x, y, and yaw coordinates, followed by a 'Saved waypoint(s) to /home/brex/outdoor_route.yaml' message. The waypoints are numbered 8 through 12. The terminal window has a title bar with 'brex@brex-desktop: ~' and standard window controls.

```
[outdoor_route_recorder-15] [INFO] [1777354636.984215203] [outdoor_route_recorder]: Recorded waypoint 8: x=11.887, y=3.850, yaw=-0.967
[outdoor_route_recorder-15] [INFO] [1777354636.916760252] [outdoor_route_recorder]: Saved 8 waypoint(s) to /home/brex/outdoor_route.yaml
[outdoor_route_recorder-15] [INFO] [1777354638.404770961] [outdoor_route_recorder]: Recorded waypoint 9: x=13.153, y=4.338, yaw=-1.802
[outdoor_route_recorder-15] [INFO] [1777354638.415592005] [outdoor_route_recorder]: Saved 9 waypoint(s) to /home/brex/outdoor_route.yaml
[outdoor_route_recorder-15] [INFO] [1777354640.403622340] [outdoor_route_recorder]: Recorded waypoint 10: x=14.818, y=5.112, yaw=-0.866
[outdoor_route_recorder-15] [INFO] [1777354640.416173469] [outdoor_route_recorder]: Saved 10 waypoint(s) to /home/brex/outdoor_route.yaml
[outdoor_route_recorder-15] [INFO] [1777354642.403627206] [outdoor_route_recorder]: Recorded waypoint 11: x=16.447, y=5.875, yaw=-1.040
[outdoor_route_recorder-15] [INFO] [1777354642.414653815] [outdoor_route_recorder]: Saved 11 waypoint(s) to /home/brex/outdoor_route.yaml
[outdoor_route_recorder-15] [INFO] [1777354644.403846279] [outdoor_route_recorder]: Recorded waypoint 12: x=17.848, y=6.401, yaw=-0.802
[outdoor_route_recorder-15] [INFO] [1777354644.416693275] [outdoor_route_recorder]: Saved 12 waypoint(s) to /home/brex/outdoor_route.yaml
```

Figure 26: Terminal showing the waypoints being passed one by one to the robot

In autonomous mode, the robot used a breadcrumb following system, or waypoint following system. The previously recorded waypoint file was loaded into the navigation stack, and Nav2 used the robot's localization estimate and costmap data to move toward each waypoint sequentially. This allowed the robot to replay previously driven routes autonomously.

4.5 Rolling Costmap and Incremental Navigation

During testing, issues were encountered when using a static map or fixed planning region for outdoor navigation. Once the robot moved outside the useful bounds of a static map, Nav2 had difficulty planning longer routes because the robot was restrained to a limited planning area. This reduced the reliability of waypoint following over larger outdoor distances.

To solve the issue, a rolling costmap configuration was implemented. Instead of the use of a fixed area map, the costmap continuously updates around the robot's current position while it moves. This allows the robot to continue planning movement beyond the original map boundaries and makes outdoor waypoint replay more reliable.

Another improvement was the implementation of an incremental route follower. Instead of sending the entire route to Nav2 at once, the system loaded the waypoint file and sent waypoints

one at a time. This simplified route replay and allowed the robot to focus on reaching smaller local goals while the rolling costmap continuously updated around the robot.

```
with open(self.route_file, 'r', encoding='utf-8') as route_stream:
    route_data = yaml.safe_load(route_stream) or {}

waypoints = route_data.get('waypoints', [])
```

Figure 27: Route File loading in the Route Follower; located in incremental_route_follower

Additional waypoint selection logic was also implemented so the robot could begin navigation near the closest waypoint rather than requiring it to start exactly at the first point in the route. This made the route recovery better and made the navigation system more flexible when testing.

```
distance = self.pose_distance(robot_pose, pose)

if distance < nearest_distance:
    nearest_distance = distance
    nearest_index = index
```

Figure 28: Closest Waypoint Selection Logic; located in outdoor_incremental_route_follower.py

4.6 Testing and Validation

Outdoor testing was performed in order to validate localization accuracy and autonomous route replay. During testing, the robot successfully recorded waypoint routes while operating in manual mode and later replayed those routes autonomously using the breadcrumb navigation system.

```
waypoints:  
- x: -0.014  
  y: 0.013  
  yaw: -0.001  
  frame_id: map  
- x: 0.488  
  y: 0.300  
  yaw: -0.506  
  frame_id: map
```

Figure 29: Waypoint route file; located in outdoor_route.yaml

The rolling costmap configuration improved the robot's ability to navigate over larger outdoor paths while maintaining a continuously updated planning region around the robot. Localization data from the EKF remained stable throughout testing and allowed Nav2 to continuously estimate the robot's position while following waypoint paths.

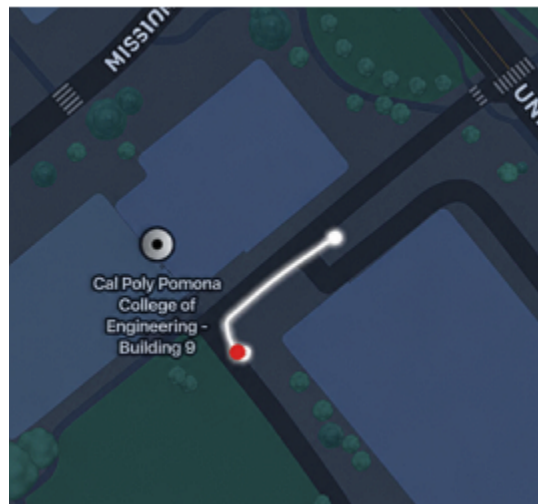


Figure 30: Overlay of the route (cost map) with Apple maps, showing path taken during testing

An overlay comparison between the robot's recorded route and Apple Maps was also generated in order to show the path taken during testing and compare the robot's localization estimate with real world positioning data. This was done by using the costmap (which was a grey and white graph with the grey showing the path the robot took) and Apple Maps (a nicer looking map).

4.7 Reflection

The autonomous navigation system successfully demonstrated localization, waypoint recording, and autonomous route replay with sensor fusion and ROS2 navigation tools. With the combination of GPS, IMU and encoder feedback through the Extended Kalman Filter, the robot was able to maintain a more stable estimate of its position and orientation during operation.

The transition to a static map to a rolling costmap approach significantly improved our outdoor navigation performance, and provided us with a more scalable solution for future autonomous testing.

5. Crash Avoidance Behavior

5.1 Semantic Segmentation and Environmental Perception

In addition to localization and autonomous navigation, the robot incorporated a perception system designed to improve environmental awareness during autonomous operation. While the localization system estimated the robot's position and movement, the perception system allowed the robot to interpret features around its environment using camera-based semantic segmentation and depth perception.



Figure 31: Semantic Segmentation of the CPP Quad with Jetson Inference

The perception system utilized an Intel RealSense stereoscopic camera, together with NVIDIA Jetson Inference, running on the Orin. Semantic segmentation was used to classify portions of the live camera feed into meaningful environmental objects such as sidewalks, grass, obstacles, and pedestrians. This provided contextual info about the robot's surroundings and established a foundation for future obstacle avoidance and autonomous path-awareness behavior.

The segmentation pipeline analyzed incoming image data and generated a color-coded mask

representing recognized environmental features. This allows for the robot to visually distinguish traversable terrain from the surrounding environment and obstacles during testing.

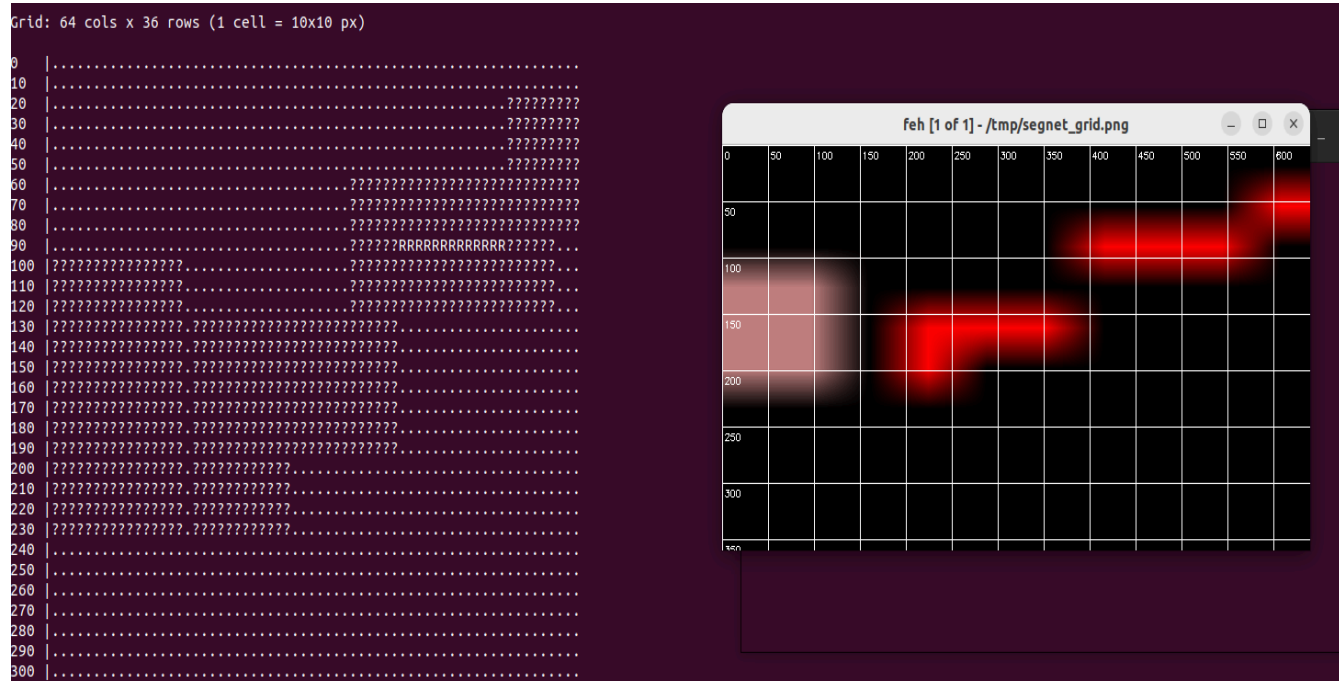


Figure 32: Custom ROS2 image topic implementation

Existing NVIDIA ROS2 integration packages were incompatible with the newer Orin and JetPack software environments used in this project. To address this, a custom ROS2 publisher implementation was made in order to publish segmentation output as ROS2 image topics. This allows other ROS2 nodes and visualization tools such as RVIZ2 to subscribe to the perception data for testing and integration purposes. The perception framework was designed to remain modular and extensible so the future spirals can integrate additional crash avoidance logic, environmental classification methods and autonomous decision-making systems into the robot.

5.2 Visualization_node

The visualization node is kept separate from the safety command path and renders a live debug overlay showing the current perception output and safety state.

5.3 Operator camera view

The node also provides a real-time camera view from the operator side, which is useful for monitoring the robot during autonomous trials.

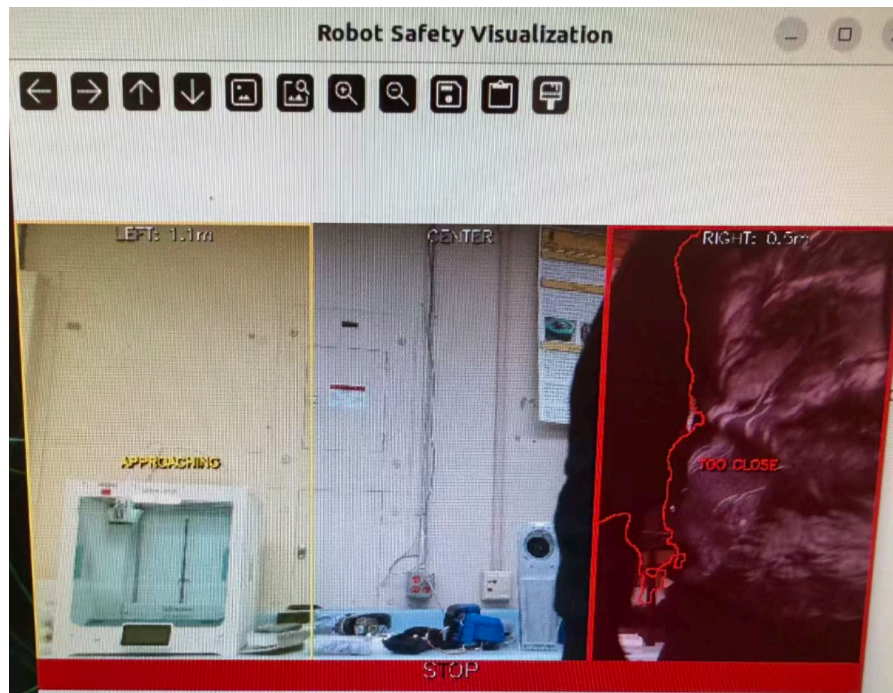


Figure 33: Operator View of the robot

5.4 Testing and integration support

A perception stub allows the safety stack to be exercised without the full camera pipeline. Launch files are provided for both full-stack operation and safety-overlay-only operation. The preferred integration method is the safety overlay launch, which lets the team's existing motor

driver remain responsible for hardware while remapping its input to /cmd_vel_safe.

Additional support components were created to improve testing, handoff, and usability. A visualization node displays a live debug overlay showing perception and safety status. A perception stub supports testing without the full camera pipeline. Launch files were prepared for both full-stack operation and overlay operation. It also provides a real time camera view from the operator side. The preferred integration method is the safety overlay launch, which allows the team's existing motor driver to remain responsible for hardware while remapping the motor input to /cmd_vel.

6. Conclusion

In conclusion, this year's brex project improved the iHerb robotic vehicle by strengthening its hardware integration, motor control system, sensor fusion pipeline, and autonomous navigation capabilities. The redesigned PCB reduced wiring issues between the motor driver and microcontroller, while the meso-level controller helped separate low-level motor execution from high-level ROS 2 processing. The robot was also updated to support both manual joystick control and autonomous `/cmd_level` commands, improving its ability to transition towards autonomous operation.

Through the use of GPS, IMU, and encoder feedback, the project developed a localization and sensor fusion system that provided a more reliable estimate of the robot's position and movement. These improvements created the foundation for autonomous route replay, waypoint navigation, and safer testing through crash avoidance behavior. Overall, the project built upon previous team developments and provided a stronger, more reliable platform for future BREx teams to continue advancing the iHerb robot toward full autonomy.

Appendix B. Works Cited

R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14-23, Mar. 1986, doi: 10.1109/JRA.1986.1087032

ROS.org, “geometry_msgs/Twist Message,” ROS Documentation. [Online]. Available: https://docs.ros.org/en/noetic/api/geometry_msgs/html/msg/Twist.html. [Accessed: April, 2026].

S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System2: Design, architecture, and uses in the wild,” **Science Robotics**, vol. 7, no. 66, p. eabm6074, May 2022, doi: 10.1126/scirobotics.abm6074.

Open Robotics, “ROS 2 Documentation: Humble Hawksbill,” ROS 2 Documentation. [Online]. Available: <https://docs.ros.org/en/humble/index.html>. [Accessed: May 11, 2026].

Intel Corporation, “Intel® RealSense™ D400 Series Product Family Datasheet,” Intel, document no. 337029. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/841984/intel-realsense-d400-series-product-family-datasheet.html>. [Accessed: May 11, 2026].

Intel Corporation, “Intel® RealSense™ Depth Camera D455 Product Specifications.” [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/205847/intel-realsense-depth-camera-d455/specifications.html>. [Accessed: May 11, 2026].

NVIDIA Corporation, “Jetson AGX Orin Developer Kit User Guide,” NVIDIA Developer. [Online]. Available: <https://developer.nvidia.com/embedded/learn/jetson-agx-orin-devkit-user-guide/index.html>. [Accessed: May 11, 2026].

STMicroelectronics, “NUCLEO-F767ZI,” STMicroelectronics. [Online]. Available: <https://www.st.com/en/evaluation-tools/nucleo-f767zi.html>. [Accessed: May 11, 2026].

STMicroelectronics, “STM32 Nucleo-144 Boards (MB1137) User Manual,” UM1974. [Online]. Available: https://www.st.com/resource/en/user_manual/um1974-stm32-nucleo144-boards-mb1137-stmicroelectronics.pdf. [Accessed: May 11, 2026].

G. Bradski, “The OpenCV Library,” **Dr. Dobb’s Journal of Software Tools**, 2000.

OpenCV, “OpenCV: Open Source Computer Vision Library.” [Online]. Available: <https://opencv.org/>. [Accessed: May 11, 2026].

C. R. Harris et al., “Array programming with NumPy,” **Nature**, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

C. Liechti, “pySerial Documentation,” pySerial. [Online]. Available: <https://pyserial.readthedocs.io/en/latest/pyserial.html>. [Accessed: May 11, 2026].

ISO, *ISO 3691-4:2023, Industrial trucks — Safety requirements and verification —Part 4: Driverless industrial trucks and their systems*, International Organization for Standardization, 2023. [Online]. Available: <https://www.iso.org/standard/83545.html>. [Accessed: May 11, 2026].

STMicroelectronics, “*Nucleo-F767ZI Development Board Documentation*,” ST.com. Available: <https://www.st.com/en/evaluation-tools/nucleo-f767zi.html>. Accessed: Feb. 2026.

Texas Instruments, “*DRV8353RH-EVM Evaluation Module Documentation*,” TI.com. Available: <https://www.ti.com/tool/DRV8353RH-EVM>. Accessed: Feb. 2026.

Altium Designer, “*Altium Designer – Official YouTube Channel*,” YouTube. Available: <https://www.youtube.com/@altiumdesigner863/featured>. Accessed: Mar. 2026

Phil’s Lab, “*KiCad Tutorial – Creating Planes*,” YouTube. Available: <https://www.youtube.com/watch?v=V9maU5P3Kqo>. Accessed: Mar. 2026.

Texas Instruments. DRV8353Rx-EVM User’s Guide. Texas Instruments, Feb 2026
STMicroelectronics. UM1974: STM32 Nucleo-144 Boards (MB1137) User Manual.
STMicroelectronics, Feb 2026

STMicroelectronics. STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx Datasheet.
STMicroelectronics, Feb 2026

KiCad. Getting Started in KiCad. KiCad Documentation, version 10.0. Apr 2026
All About Circuits. “3-Phase Brushless DC Motor Control with Hall Sensors.” All About Circuits, 26 June 2018. Feb 2026

NXP Semiconductors. 3-Phase BLDC Motor Control with Hall Sensors Using the 56F8013.
NXP Semiconductors. Feb 2026

STMicroelectronics. PWM Management for 3-Phase BLDC Motor Drives Using the ST7MC.
STMicroelectronics. Feb 2026

dusty-nv, jetson-inference. GitHub. Available:<https://github.com/dusty-nv/jetson-inference>.
Accessed: Apr. 25, 2026.

dusty-nv, ros_deep_learning. GitHub. Available:https://github.com/dusty-nv/ros_deep_learning.
Accessed: Apr. 25, 2026.

ros2, common_interfaces. GitHub.
Available:https://github.com/ros2/common_interfaces/blob/humble/sensor_msgs/msg/Image.msg .
Accessed: Apr. 25, 2026.

ros-navigation, navigation2. GitHub. Available: <https://github.com/ros-navigation/docs.nav2.org>.
Accessed: Apr. 25, 2026.

D. Dawson (Dusty-NV), “Jetson inference: Hello AI world,” GitHub repository. [Online].

Available: <https://github.com/dusty-nv/jetson-inference>. Accessed: Apr. 2025.

ROS Developers, “joy: Generic joystick driver,” ROS 2 Documentation, ver. 3.3.0, 2024. [Online]. Available: <https://index.ros.org/p/joy/>. Accessed: Apr. 2025.